
Tree Ensembles: Random Forests & Boosted Trees

ECON 8310: Business Forecasting — Lecture 5

Department of Economics
University of Nebraska at Omaha
August 25, 2026

Lecture Outline

- 1 From Single Trees to Ensembles
- 2 Bagging: Bootstrap Aggregating
- 3 Random Forests: Decorrelating the Trees
- 4 Feature Importance and Interpretation
- 5 Gradient Boosting
- 6 XGBoost
- 7 Key Takeaways and Roadmap

From Single Trees to Ensembles

Averaging many imperfect models can outperform any single model.

The Wisdom of Crowds

A single expert can be wrong. Average many *independent* estimates and the errors tend to cancel.

Classic Example: Guessing the Ox's Weight

At a 1907 county fair, 800 people guessed the weight of an ox. No individual was very accurate — but the **average of all 800 guesses** came within one pound of the true weight.

The same principle transfers directly to models. A single decision tree is unstable and noisy — that was Lecture 4's closing complaint. Train many trees on different versions of the data and average them, and the noise largely cancels while the signal survives.

Ensembles work because independent errors cancel. And that word is doing real work: the *more independent* the individual predictions, the more variance averaging removes. Hold onto it — the entire difference between bagging and random forests turns on it.

Bagging: Bootstrap Aggregating

Train many trees on bootstrap samples; average their predictions.

Bagging: The Big Idea

Problem from Lecture 4: a single decision tree has very high variance — change a few training points and you get a completely different tree.

Bagging solution (Breiman 1996):

1. Draw B **bootstrap samples** (with replacement, same size as the original)
2. Fit one deep decision tree on each
3. **Average** all B predictions: $\hat{y}_{\text{bag}} = \frac{1}{B} \sum_{b=1}^B \hat{f}^{(b)}(\mathbf{x})$

Why it works. The variance of the averaged forecast is

$$\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2,$$

where ρ is the pairwise correlation between tree predictions. The second term shrinks as B grows — so more trees never hurts. But the first term does not move at all.

That leaves a floor at $\rho\sigma^2$, set by how alike the trees are rather than by how many you grow. Averaging a thousand nearly identical trees buys you almost nothing, and that is the problem the next section solves.

Out-of-Bag (OOB) Error: Free Cross-Validation

Each bootstrap sample draws n observations *with replacement*, so it contains about **63%** of the distinct training points. The remaining 37% are **out-of-bag** for that tree — a free test set, different for every tree:

1. For each observation i , collect predictions only from the trees where i was out-of-bag
2. Average them to get $\hat{y}_i^{\text{OOB}}$
3. Compute the RMSE across all observations

Every one of those predictions came from a tree that never saw the observation — so the OOB RMSE is an honest out-of-sample estimate, and it lands close to 5-fold CV without refitting anything. In `sklearn` it is one argument:

`RandomForestRegressor(oob_score=True)`.

Below about 1,000 observations the OOB estimate gets noisy, because each observation is out-of-bag for too few trees. On a weekly business series — a few hundred rows — use proper walk-forward CV instead.

Random Forests: Decorrelating the Trees

Restrict each split to a random subset of features so trees are less alike.

The Problem Bagging Does Not Fully Solve

Recall the variance of the averaged forecast: $\text{Var}(\bar{f}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$. More trees shrink the second term to nothing. The first term never moves.

The correlation problem. If one feature dominates — and in forecasting it always does, because last year's value is the best single predictor — then nearly every tree splits on it at the root. The trees end up near-identical, ρ stays large, and bagging buys you very little.

The Random Forest fix (Breiman 2001) is almost absurdly simple: at each split, consider only a random subset of $m < p$ features, and take the best split among *those*.

Sometimes the dominant feature is not in the candidate set, so the tree is forced to find its second-best story. Across hundreds of trees that produces genuinely different structures — ρ falls, and the floor falls with it. You deliberately make each individual tree *worse* in order to make the ensemble better.

Random Forest Hyperparameters

Parameter	sklearn name	Default	Effect
Number of trees	<code>n_estimators</code>	100	More = lower variance; plateaus ~ 500
Max features/split	<code>max_features</code>	'sqrt'	Lower = more decorrelated
Max depth	<code>max_depth</code>	None	Limit only to reduce memory
Min samples/leaf	<code>min_samples_leaf</code>	1	Higher = smoother predictions
OOB score	<code>oob_score</code>	False	True gives a free CV estimate

Start with `n_estimators=500`, `max_features='sqrt'`, `min_samples_leaf=5`, `oob_score=True`. Note how few genuinely need tuning — `max_features` is the dial controlling ρ , and the rest is close to fire-and-forget. That is most of why random forests make a good default.

Set `random_state=42`: a forest is random twice over, in the bootstrap samples and in the feature subsets.

Feature Importance and Interpretation

Which predictors matter most? RF gives a data-driven answer.

Random Forest Feature Importance (Molnar 2022)

Mean Decrease in Impurity (MDI)

For each feature j , average the total reduction in node impurity across every split on j , weighted by the fraction of observations reaching that node. Normalize to sum to 1.

Feature (CA_1 FOODS, weekly)	Importance
units_lag4	0.45
units_lag1	0.18
units_lag52	0.11
snap_days	0.08
units_lag2	0.04
avg_price	0.04

Read it as a screening device, not an explanation. **MDI is biased** toward features with many split points. `permutation_importance` — shuffle a column, see how much accuracy falls — is the honest alternative.

Random Forest in Python

sklearn Code Template

```
from sklearn.ensemble import RandomForestRegressor  
rf = RandomForestRegressor(n_estimators=500, max_features='sqrt',  
min_samples_leaf=5, oob_score=True, random_state=42).fit(X_tr, y_tr)  
print(rf.oob_score_); importances = rf.feature_importances_
```

Model (CA_1 FOODS, 52-week test)	Test RMSE
Seasonal naïve	1,660
Single deep tree	1,415
Random Forest	1,020

The forest cuts RMSE roughly **28%** against a single deep tree, and is the first method in this course to beat the seasonal naïve benchmark on this series.

What changed is not a better model of the series but a richer feature set — lags, price, and SNAP days are information ETS and ARIMA never had.

Gradient Boosting

Instead of averaging independent trees, fit each new tree to the errors of all previous trees.

Boosting vs. Bagging: The Core Difference

Bagging (Random Forest):

- Trees built **in parallel**, independently
- Each tree tries to predict y directly
- Average reduces variance
- Adding more trees does not reduce bias

Boosting:

- Trees built **sequentially**
- Each tree fits the *errors* of the previous ensemble
- Adding more trees reduces both bias AND variance (carefully)
- Risk: can overfit if trees are too deep or too many

Analogy: Correcting a Forecast Iteratively

Forecast 1: "Sales will be \$50,000." Actual: \$60,000. Error: +\$10,000.

Forecast 2: "The first model was short by \$10,000."

Combined prediction: $\$50,000 + \$10,000 = \mathbf{\$60,000}$.

Each stage corrects the remaining error from all previous stages.

The Gradient Boosting Algorithm

Gradient Boosting (for squared-error loss)

Initialize $F_0(\mathbf{x}) = \bar{y}$. For $b = 1, \dots, B$:

1. Compute residuals: $r_i^{(b)} = y_i - F_{b-1}(\mathbf{x}_i)$
2. Fit a shallow tree T_b to $\{(\mathbf{x}_i, r_i^{(b)})\}$
3. Update: $F_b = F_{b-1} + \eta T_b$

Final prediction: $\hat{y} = F_B(\mathbf{x})$

The learning rate η :

- Small η (0.01–0.1): tiny step per tree — requires more trees but more robust
- Large η (> 0.3): fast but risks overshooting
- **Rule:** smaller η + more trees + early stopping typically wins

Trees are **shallow** (depth 3–6). Each corrects a small fraction of the total error. The ensemble slowly converges.

Warning: always use **early stopping** — without it, boosting will overfit.

XGBoost

Three improvements over vanilla boosting: second-order optimization, explicit regularization, and column subsampling.

XGBoost: Three Advances (Chen and Guestrin 2016)

1. **Newton step (2nd-order optimization):**

Uses the gradient *and* curvature (Hessian) of the loss function to compute more precise tree-fitting directions. Converges faster than first-order boosting.

2. **Explicit regularization:** Penalizes the number of leaves (γ) and the magnitude of leaf weights (λ) directly in the objective. Guards against overfitting without relying on early stopping alone.

3. **Column subsampling:** Like Random Forests, randomly samples features per tree and per split. Reduces correlation and improves generalization.

XGBoost dominated Kaggle competitions from 2014–2018. It consistently outperforms vanilla gradient boosting on tabular data.

Other popular options:

- LightGBM — faster on large datasets
- CatBoost — handles categorical features natively

XGBoost: Key Hyperparameters

Parameter	XGB name	Typical	Effect
Number of trees	<code>n_estimators</code>	500–2000	More + small η = better (use early stopping)
Learning rate	<code>learning_rate</code>	0.01–0.1	Smaller = more robust; needs more trees
Tree depth	<code>max_depth</code>	3–6	Shallow preferred; depth >6 risks overfit
Row subsample	<code>subsample</code>	0.7–0.9	Fraction of training rows per tree
Col. subsample	<code>colsample_bytree</code>	0.7–0.9	Feature fraction per tree (like RF)
L2 leaf penalty	<code>reg_lambda</code>	1	Ridge on leaf weights; stabilizes

Practical starting point: `learning_rate=0.05`, `max_depth=4`, `n_estimators=1000` with early stopping on validation RMSE. Tune `max_depth` and `subsample` via `TimeSeriesSplit`.

Code Template

```
model = xgb.XGBRegressor(  
    n_estimators=1000,  
    learning_rate=0.05,  
    max_depth=4,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    early_stopping_rounds=50,  
    random_state=42)  
model.fit(  
    X_train, y_train,  
    eval_set=[(X_val, y_val)],  
    verbose=False)
```

Cross-validation for time series:

- Use `TimeSeriesSplit` with the sklearn API
- Reserve last 20% as final test set
- `early_stopping_rounds=50`: stops if val. RMSE stalls for 50 rounds

Typical RSXFS result: XGBoost RMSE $\approx$ 1,500–1,700 (vs. RF $\approx$ 1,800).

Key Takeaways and Roadmap

Bagging reduces variance; boosting reduces bias. Together they cover most of what tabular forecasting needs.

Tree Methods: Which One When

Method	Training	Interpretable	Tuning	Typical accuracy
Decision Tree	Single tree	High	Low	Baseline
Random Forest	Parallel (B trees)	Medium	Low	Good
XGBoost	Sequential (B trees)	Low	Higher	Best

Random Forest strengths:

- ✓ Robust — hard to overfit badly
- ✓ Minimal tuning; good out-of-the-box
- ✓ OOB error gives free cross-validation

Shared limitation:

- ✗ Neither can extrapolate beyond the training range — a hard constraint for trending series

Practical order: start with a Random Forest — it is hard to make catastrophically wrong. Move to XGBoost when you have the time to tune and need the last few points of accuracy.

Lecture 5: Key Takeaways

1. **Ensembles** work because independent errors cancel. Averaging many imperfect models beats any single model.
2. **Bagging** averages B trees fit to bootstrap samples; variance falls as B grows but floors at $\rho\sigma^2$. **Random Forests** subsample features at each split to cut ρ — the key gain over plain bagging.
3. **OOB error** is free cross-validation: each observation sits out $\approx 37\%$ of trees, giving an unbiased generalization estimate.
4. **Boosting** inverts the idea — trees are built sequentially, each fitting the ensemble's remaining errors. Bagging attacks variance; boosting attacks bias. **XGBoost** adds a Newton step, penalties on leaf count and weights, and column subsampling.
5. Tune by cross-validation, always with `TimeSeriesSplit` — a random split leaks the future into the past — and pair boosting with **early stopping**.

Next: Regularization & Model Selection — choosing among many predictors by shrinking coefficients rather than counting them.

References I

-  Breiman, Leo (1996). “Bagging Predictors”. In: *Machine Learning* 24.2, pp. 123–140.
-  — (2001). “Random Forests”. In: *Machine Learning* 45.1, pp. 5–32.
-  Chen, Tianqi and Carlos Guestrin (2016). “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794.
-  Molnar, Christoph (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 2nd. Open access. URL: <https://christophm.github.io/interpretable-ml-book/>.